

Contents

Musterlösungen — NUR für den Dozenten	1
Epic 1	1
Epic 2	3
Epic 3	5
Epic 4	7
Epic 5	8

Musterlösungen — NUR für den Dozenten

Nicht an Schüler weitergeben, bevor sie es selbst versucht haben. Lösungen sind bewusst einfach gehalten (Anfänger-Niveau), nicht “optimal” im Sinne von cleverem Kotlin — Ziel ist Nachvollziehbarkeit, nicht Eleganz.

Noch geradlinigere Variante: [loesungen_easy.md](#) zeigt dieselben Storys mit ausführlichem if/else, for-Schleifen statt minByOrNull und ohne ausgelagerte Hilfsfunktionen — näher an dem, was Anfänger tatsächlich schreiben. Gut als realistischer Review-Maßstab und für Teams, die bei der kompakten Version aussteigen.

Alle Beispiele gehen von `import framework.arena.*` aus. Hilfsfunktionen (z.B. Distanzberechnung), die in mehreren Lösungen wiederkehren, sind bei der ersten Verwendung ausgeschrieben und danach nur noch referenziert — echte Referenzimplementierungen dieser Muster liegen bereits fertig in `bots/examples/` (RandomBot, StillstandBot, ChaserBot, FluchtBot, PowerBot), auf die man Schüler bei Bedarf verweisen kann, ohne die Lösung direkt zu zeigen.

Epic 1

1.1 — Zufällige Bewegung

```
class MeinBot(override val name: String = "MeinBot") : RobotBrain {  
    override fun decide(sensors: Sensors): Action {  
        return Action.Move(Direction.entries.random())  
    }  
}
```

Entspricht im Kern `bots/examples/RandomBot.kt` (dort zusätzlich mit zufälliger Wahl zwischen Move/Shoot).

Erklärung: - `decide()` wird von der Engine jeden Tick genau einmal aufgerufen und muss eine `Action` zurückgeben — hier immer `Action.Move(...)`. - `Direction.entries` ist die Kotlin-Standardmethode, um alle Werte eines enum class als Liste zu bekommen (NORTH, EAST, SOUTH, WEST). `.random()` wählt daraus zufällig einen Wert. - Intention: zeigen, dass der Bot überhaupt korrekt eingebunden ist (Akzeptanzkriterium aus Story 1.1) — es geht hier noch nicht um Strategie, sondern nur darum, dass `decide()` syntaktisch korrekt eine `Action` liefert und der Bot in der UI sichtbar herumläuft. - Typischer Anfängerfehler: `Direction.values()` statt `.entries` — funktioniert auch, aber `.entries` ist der modernere, empfohlene Weg in Kotlin 1.9+ (weniger Object-Allokation). Beides als richtig durchgehen lassen.

1.2 — Am Rand bleiben / Zentrum meiden

```
class MeinBot(override val name: String = "MeinBot") : RobotBrain {  
    override fun decide(sensors: Sensors): Action {  
        val centerX = sensors.arenaWidth / 2
```

```

val centerY = sensors.arenaHeight / 2
val pos = sensors.self.position

val nearCenter = kotlin.math.abs(pos.x - centerX) <= 1 && kotlin.math.abs(pos.y - centerY) <= 1
if (nearCenter) {
    // Richtung mit dem größeren Abstand zum jeweils nächsten Rand wählen
    return if (pos.x < centerX) Action.Move(Direction.WEST) else Action.Move(Direction.EAST)
}
return Action.Move(Direction.entries.random())
}
}

```

Erklärung: - `sensors.arenaWidth / 2` und `sensors.arenaHeight / 2` liefern die Mitte des 10×10-Grids (bei ungeraden Größen abgerundet — hier irrelevant, da 10 gerade ist). Ganzzahldivision (`Int / Int`) reicht, kein `.toDouble()` nötig. - `pos` ist die eigene aktuelle Position (`sensors.self.position`), ein `Position(x, y)`-Datenobjekt. - `nearCenter` prüft, ob der Bot innerhalb eines 3×3-Feldes um die Mitte steht (`abs(...) <= 1` in beide Achsen). `kotlin.math.abs` ist der voll qualifizierte Aufruf — man könnte auch `import kotlin.math.abs` oben ergänzen und dann nur `abs(...)` schreiben; beides ist gleichwertig, hier bewusst ausgeschrieben, damit Schüler den Import nicht vermissen, wenn sie den Code direkt kopieren. - Ist der Bot zu nah an der Mitte, bewegt er sich Richtung des am weitesten entfernten Randes auf der X-Achse (WEST, wenn er links von der Mitte steht, sonst EAST). Das ist bewusst vereinfacht — nur eine Achse wird betrachtet, nicht die “wirklich nächste” Ecke. Für Anfänger ok; wer will, kann analog auch die Y-Achse einbeziehen (Kür). - Ist der Bot nicht in Zentrumsnähe, verhält er sich wie 1.1 (Zufallsbewegung) — zeigt Schülern, wie sich Lösungen aus früheren Storys wiederverwenden/kombinieren lassen, statt alles neu zu schreiben.

1.3 — Flucht bei niedriger Gesundheit

```

class MeinBot(override val name: String = "MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val nearestEnemy = sensors.others.minByOrNull { distance(sensors.self.position, it) }
        ?: return Action.Wait

        if (sensors.self.health < 20) {
            // Weg vom Gegner: Richtung mit größerem Abstandszuwachs wählen
            val dx = sensors.self.position.x - nearestEnemy.position.x
            val dy = sensors.self.position.y - nearestEnemy.position.y
            return if (kotlin.math.abs(dx) > kotlin.math.abs(dy)) {
                Action.Move(if (dx > 0) Direction.EAST else Direction.WEST)
            } else {
                Action.Move(if (dy > 0) Direction.SOUTH else Direction.NORTH)
            }
        }

        // Normalverhalten, z.B. aus Epic 2 (hier: einfach draufzu bewegen)
        val dx = nearestEnemy.position.x - sensors.self.position.x
        val dy = nearestEnemy.position.y - sensors.self.position.y
        return if (kotlin.math.abs(dx) > kotlin.math.abs(dy)) {
            Action.Move(if (dx > 0) Direction.EAST else Direction.WEST)
        } else {
            Action.Move(if (dy > 0) Direction.SOUTH else Direction.NORTH)
        }
    }
}

```

```
}
```

```
private fun distance(a: Position, b: Position): Int =  
    kotlin.math.abs(a.x - b.x) + kotlin.math.abs(a.y - b.y)
```

Erklärung: - distance() berechnet die sog. Manhattan-Distanz (Summe der Achsenabstände), nicht die euklidische Luftlinie — passend zum Grid, auf dem sich Bots nur horizontal/vertikal bewegen (keine Diagonalen). Diese Hilfsfunktion taucht ab hier in fast jeder Lösung wieder auf; sie steht schon fertig in bots/examples/ChaserBot.kt und FluchtBot.kt. - sensors.others.minByOrNull { distance(...) } sucht unter allen noch lebenden Gegnern (sensors.others enthält laut API nur lebende Roboter) denjenigen mit der kleinsten Distanz. minByOrNull statt minBy, weil die Liste leer sein kann (letzter Gegner tot) — dann liefert es null statt eine Exception zu werfen. Der ?: return Action.Wait fängt genau diesen Fall ab (siehe Akzeptanzkriterium in Story 1.3: “keine Exception werfen”). - Kernlogik der Flucht: dx/dy ist der Vektor vom Gegner zum eigenen Bot (nicht umgekehrt!) — daher bewegt sich der Bot in die Richtung, die diesen Abstand vergrößert. Es wird nur die Achse mit dem größeren Ausschlag bewegt (abs(dx) > abs(dy)), weil Action.Move nur eine Richtung pro Tick erlaubt — man kann nicht diagonal in einem Schritt fliehen. - Ist dx > 0 (Gegner liegt westlich vom Bot), muss der Bot nach Osten fliehen — daher if (dx > 0) EAST else WEST. Diese Vorzeichenlogik ist erfahrungsgemäß die Stelle, an der Schüler sich am häufigsten vertun (Richtung “zum Gegner hin” vs. “vom Gegner weg” verwechseln); beim Review genau gegenprüfen. - Unterhalb der Gesundheitsschwelle wird exakt dieselbe Bewegungslogik nochmal verwendet, nur mit vertauschtem Vorzeichen (Verfolgen statt Fliehen) — im finalen Bot würde man das in eine gemeinsame Hilfsfunktion auslagern; hier bewusst dupliziert, um die Lösung ohne zusätzliche Abstraktion lesbar zu halten (Anfänger-Niveau).

Vollständige, robustere Variante liegt bereits fertig in bots/examples/FluchtBot.kt.

Epic 2

2.1 — Dauerfeuer in feste Richtung

```
class MeinBot(override val name: String = "MeinBot") : RobotBrain {  
    override fun decide(sensors: Sensors): Action {  
        return Action.Shoot(Direction.EAST)  
    }  
}
```

Erklärung: - Einfachster mögliche Bot mit Action.Shoot: feste Richtung, keine Sensorik nötig. Zeigt Schülern die zweite Hälfte der Action-API (nach Move in Epic 1). - Damit sichtbar Schaden entsteht (Akzeptanzkriterium), muss der Gegner zufällig in der Schusslinie stehen — daher der Test gegen StillstandBot, der sich nicht bewegt und so eine kontrollierbare Testsituation schafft.

Entspricht bots/examples/StillstandBot.kt.

2.2 — Auf nächsten Gegner in Sichtlinie zielen

```
class MeinBot(override val name: String = "MeinBot") : RobotBrain {  
    override fun decide(sensors: Sensors): Action {  
        val self = sensors.self.position  
  
        val inLineOfSight = sensors.others.filter { it.position.x == self.x || it.position.y == self.y }  
        val target = inLineOfSight.minByOrNull { distance(self, it.position) } ?: return Action.Wait  
  
        val direction = when {  
            target.x < self.x -> Direction.WEST  
            target.x > self.x -> Direction.EAST  
            target.y < self.y -> Direction.NORTH  
            target.y > self.y -> Direction.SOUTH  
            else -> Direction.WEST  
        }
```

```

        target.position.x == self.x && target.position.y < self.y -> Direction.NORTH
        target.position.x == self.x && target.position.y > self.y -> Direction.SOUTH
        target.position.y == self.y && target.position.x > self.x -> Direction.EAST
    else -> Direction.WEST
}
return Action.Shoot(direction)
}
}

```

```

private fun distance(a: Position, b: Position): Int =
    kotlin.math.abs(a.x - b.x) + kotlin.math.abs(a.y - b.y)

```

Erklärung: - “In Sichtlinie” heißt hier: gleiches x (Gegner steht in derselben Spalte, oben oder unten) oder gleiches y (gleiche Reihe, links oder rechts) — auf dem Grid gibt es keine diagonale Schusslinie, nur die 4 Grundrichtungen. - `sensors.others.filter { ... }` reduziert zuerst auf alle Kandidaten, die überhaupt treffbar sind, dann wählt `minByOrNull { distance(...) }` unter diesen den nächsten aus. Wichtig: Filtern *vor* dem Sortieren nach Distanz, sonst würde man evtl. auf den nächsten Gegner insgesamt zielen, der aber gar nicht in Reihe/Spalte steht. - Der `when`-Block leitet aus der *relativen Position* des Ziels die Schussrichtung ab. Reihenfolge der Bedingungen ist bewusst so gewählt, dass `x == self.x` zuerst geprüft wird (Ziel oben/unten), erst danach `y == self.y` (Ziel links/rechts) — bei exakt gleicher Position (kann durch Kollisionsregeln der Engine nicht vorkommen) wäre das Verhalten sonst nicht eindeutig. - `target.position.y < self.y ->` Ziel liegt “weiter oben” -> NORTH. Wichtig für Schüler: die Y-Achse wächst nach **unten** (wie bei Bildschirmkoordinaten, nicht wie im Mathe-Koordinatensystem) — das ist eine häufige Verwechslungsquelle, die man beim Review explizit ansprechen sollte. - Kein Kandidat in Sichtlinie -> `Action.Wait` als Platzhalter (Akzeptanzkriterium: nicht sinnlos ins Leere schießen). In Kombination mit 2.3 wird dieser Fall stattdessen zur Verfolgung genutzt.

2.3 — Gegner verfolgen, wenn nicht in Schusslinie

Kombiniert mit 2.2 — das ist im Kern die Logik von `bots/examples/ChaserBot.kt`:

```

class MeinBot(override val name: String = "MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val self = sensors.self.position
        if (sensors.others.isEmpty()) return Action.Wait

        val nearest = sensors.others.minByOrNull { distance(self, it.position) }!!

        val inLineOfSight = nearest.position.x == self.x || nearest.position.y == self.y
        if (inLineOfSight) {
            val direction = when {
                nearest.position.x == self.x && nearest.position.y < self.y -> Direction.NORTH
                nearest.position.x == self.x && nearest.position.y > self.y -> Direction.SOUTH
                nearest.position.y == self.y && nearest.position.x > self.x -> Direction.EAST
                else -> Direction.WEST
            }
            return Action.Shoot(direction)
        }

        // Nicht in Sichtlinie: Richtung mit größerer Distanz zuerst angleichen
        val dx = nearest.position.x - self.x
        val dy = nearest.position.y - self.y
        return if (kotlin.math.abs(dx) > kotlin.math.abs(dy)) {
            Action.Move(if (dx > 0) Direction.EAST else Direction.WEST)
        }
    }
}

```

```

    } else {
        Action.Move(if (dy > 0) Direction.SOUTH else Direction.NORTH)
    }
}
}

```

```

private fun distance(a: Position, b: Position): Int =
    kotlin.math.abs(a.x - b.x) + kotlin.math.abs(a.y - b.y)

```

Erklärung: - Zeigt die zentrale Kombinationslogik von Epic 2: pro Tick wird zuerst geprüft “kann ich gerade treffen?” (Sichtlinie), erst wenn nein, wird bewegt statt geschossen. Diese if/else-Weiche (inLineOfSight ja/nein) ist die Vorstufe zur Zustandsmaschine in Epic 3. - sensors.others.isEmpty() wird hier vor der minByOrNull-Suche geprüft statt danach mit ?: — funktional identisch zu 1.3/2.2, nur anders geschrieben, damit Schüler beide Stile kennenlernen (früher Return vs. Elvis-Operator). Danach ist das !! bei minByOrNull { ... }!! sicher, weil die Leerheit schon ausgeschlossen wurde — !! ohne diese Vorprüfung wäre ein Crash-Risiko und sollte im Review als Red Flag behandelt werden, wenn die Prüfung fehlt. - Bewegungslogik im “sonst”-Zweig ist identisch zur Verfolgung aus 1.3 (nur mit umgekehrtem Vorzeichen zur Flucht dort): dx/dy zeigen vom eigenen Bot zum Gegner, die Achse mit dem größeren Ausschlag wird zuerst angeglichen. Das ist eine einfache, aber nicht optimale Heuristik — der Bot bewegt sich nicht zwangsläufig auf dem kürzesten Weg in Sichtlinie, sondern läuft “diagonal genähert” (abwechselnd X dann Y), was für Anfänger aber völlig ausreichend und nachvollziehbar ist.

Für Details und die exakte, bereits getestete Referenz: bots/examples/ChaserBot.kt ansehen.

Epic 3

3.1 — Einfache Zustandsmaschine

```

private enum class BotState { PATROUILLE, ANGRIFF, FLUCHT }

```

```

class MeinBot(override val name: String = "MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val self = sensors.self
        val nearest = sensors.others.minByOrNull { distance(self.position, it.position)

        val state = when {
            self.health < 20 && nearest != null -> BotState.FLUCHT
            nearest != null -> BotState.ANGRIFF
            else -> BotState.PATROUILLE
        }

        return when (state) {
            BotState.FLUCHT -> {
                val dx = self.position.x - nearest!!.position.x
                val dy = self.position.y - nearest.position.y
                if (kotlin.math.abs(dx) > kotlin.math.abs(dy))
                    Action.Move(if (dx > 0) Direction.EAST else Direction.WEST)
                else
                    Action.Move(if (dy > 0) Direction.SOUTH else Direction.NORTH)
            }
            BotState.ANGRIFF -> {
                val inLineOfSight = nearest!!.position.x == self.position.x || nearest.

```

```

        if (inLineOfSight) {
            val direction = when {
                nearest.position.x == self.position.x && nearest.position.y < self.position.y -> Direction.NORTH
                nearest.position.x == self.position.x -> Direction.SOUTH
                nearest.position.x > self.position.x -> Direction.EAST
                else -> Direction.WEST
            }
            Action.Shoot(direction)
        } else {
            val dx = nearest.position.x - self.position.x
            val dy = nearest.position.y - self.position.y
            if (kotlin.math.abs(dx) > kotlin.math.abs(dy))
                Action.Move(if (dx > 0) Direction.EAST else Direction.WEST)
            else
                Action.Move(if (dy > 0) Direction.SOUTH else Direction.NORTH)
        }
    }
    BotState.PATROUILLE -> Action.Move(Direction.entries.random())
}
}
}
}
}

```

```

private fun distance(a: Position, b: Position): Int =
    kotlin.math.abs(a.x - b.x) + kotlin.math.abs(a.y - b.y)

```

Erklärung: - private enum class BotState ist bewusst kein gespeichertes Feld im Bot (kein var currentState), sondern wird jeden Tick frisch aus sensors berechnet — das entspricht dem Akzeptanzkriterium “Zustand wird bei jedem decide()-Aufruf neu bestimmt”. Das macht den Bot zustandslos/robust: er reagiert immer korrekt auf die aktuelle Lage, auch wenn z.B. ein Tick durch den Shake-up-Mechanismus (siehe CLAUDE.md) übersprungen wurde. - Die drei when-Zweige bilden die Zustandsübergänge ab, in Prioritätsreihenfolge: zuerst wird geprüft, ob geflohen werden muss (health < 20), erst danach ob überhaupt ein Gegner da ist. Diese Reihenfolge ist wichtig — ohne sie würde ein fast toter Bot mit Gegner in Sichtweite lieber angreifen statt fliehen. - nearest != null wird zweimal in der state-Berechnung geprüft (in FLUCHT- und ANGRIFF-Zweig), aber danach im Body mit nearest!! entsperrt (Non-null-Assertion). Kotlin kann hier leider nicht automatisch “smart casten”, weil nearest und state zwei unabhängige vals sind — der Compiler weiß nicht, dass state == ANGRIFF bereits impliziert nearest != null. Das ist ein guter Anlass, Schülern zu zeigen, warum !! hier vertretbar ist (die Logik stellt sicher, dass es nie null ist), aber grundsätzlich mit Vorsicht zu benutzen ist. - Bewegungs-/Schusslogik in FLUCHT und ANGRIFF ist wortwörtlich aus 1.3 und 2.3 übernommen — der didaktische Punkt dieser Story ist nicht neue Logik, sondern das *Strukturieren* von bereits bekanntem Verhalten in benannte, klar abgegrenzte Zustände (bessere Lesbarkeit/Erweiterbarkeit, siehe Akzeptanzkriterium “nachvollziehbar und erweiterbar”). - BotState.PATROUILLE fällt auf die Zufallsbewegung aus 1.1 zurück, wenn kein Gegner mehr existiert (z.B. nur noch der eigene Bot lebt, oder alle Gegner tot sind — dann ist das Match ohnehin fast vorbei).

3.2 — Zielpriorisierung (schwächster Gegner zuerst)

```

class MeinBot(override val name: String = "MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        // Kriterium: niedrigste HP zuerst (leichtestes Ziel zum Ausschalten)
        val target = sensors.others.minByOrNull { it.health } ?: return Action.Wait
        // ... danach wie 2.2/2.3 in Richtung von `target` schießen/bewegen
        return Action.Wait // Platzhalter – Kombination mit 2.2/2.3 ergänzen
    }
}

```



```
}
}
```

Hinweis für Schüler: der eigentliche Lernpunkt hier ist die Auswahl-Regel (`minByOrNull { it.health }` statt `minByOrNull { distance }`), nicht die Bewegungs-/Schuss-Logik selbst — die kann 1:1 aus Epic 2 übernommen werden.

Erklärung: - `sensors.others.minByOrNull { it.health }` ersetzt einfach das Auswahlkriterium aus 1.3/2.2/2.3 (`distance(...)`) durch `it.health` — sonst identisches Prinzip: aus einer Liste anhand eines Merkmals das Minimum wählen. Zeigt, dass `minByOrNull` generisch für beliebige vergleichbare Kriterien funktioniert, nicht nur für Distanz. - Bewusst als Platzhalter (`return Action.Wait`) belassen: die Story soll nicht nochmal die komplette Schuss-/Bewegungslogik aus Epic 2 abschreiben lassen, sondern nur zeigen, dass sich `target` hier statt `nearest` in genau dieselbe Struktur wie 2.2/2.3 einsetzen lässt. Beim Review darauf achten, dass Schüler die Kombination tatsächlich vervollständigen (sonst schießt/bewegt sich der Bot nie). - Fachlicher Hintergrund für den Dozenten: “schwächster Gegner zuerst” ist eine von mehreren möglichen Heuristiken (Alternativen: nächster Gegner, meistbedrohlicher Gegner). Die Story verlangt kein bestimmtes Kriterium, nur dass eines klar benannt ist — akzeptiert daher auch andere sinnvolle Kriterien der Schüler.

3.3 — Kür-Aufgabe

Keine feste Musterlösung (bewusst offen). Bei Rückfragen: prüfen, ob die Idee mit `Sensors/Action` ausdrückbar ist (kein Bot kann z.B. “sehen”, was ein Gegner als Nächstes tun wird — nur den aktuellen `RobotState` aller anderen).

Hinweis für den Dozenten: Die `Sensors`-API liefert pro Tick nur eine Momentaufnahme (eigener Zustand + Zustand aller lebenden Gegner, keine Historie, keine Vorhersage). Typische Kür-Ideen, die gut umsetzbar sind: Bewegung entlang der Arena-Kante (Wall-Following über Positionsvergleich mit `arenaWidth/arenaHeight`), Gegner in eine Ecke drängen (Bewegung relativ zur Position mehrerer `sensors.others`), “nur schießen, wenn sicher getroffen wird” (z.B. nur schießen, wenn genau ein Gegner in Sichtlinie ist statt mehrere gleich nah). Ideen, die eine Erinnerung an vergangene Ticks brauchen (z.B. “merke dir, wohin der Gegner zuletzt gelaufen ist”), sind ebenfalls möglich über eine `var`-Property im Bot — sollte aber vorher kurz mit dem Team besprochen werden, da das über das bisher Gelernte hinausgeht.

Epic 4

4.1 — Unit-Test für eigene Entscheidungslogik

```
package bots.teama
```

```
import framework.arena.*
```

```
import kotlin.test.Test
```

```
import kotlin.test.assertEquals
```

```
class MeinBotTest {
```

```
    @Test
```

```
    fun `schießt wenn Gegner in gleicher Reihe steht`() {
```

```
        val self = RobotState(id = "bot-0", teamName = "Team A", position = Position(2,
```

```
        val enemy = RobotState(id = "bot-1", teamName = "Team B", position = Position(7
```

```
        val sensors = Sensors(self = self, others = listOf(enemy), arenaWidth = 10, are
```

```
        val action = MeinBot().decide(sensors)
```

```

        assertEquals(Action.Shoot(Direction.EAST), action)
    }
}

```

Wichtig: Test-Datei muss unter `src/test/kotlin/bots/teamX/...` liegen (Ordner ggf. neu anlegen), damit Gradle sie findet.

Erklärung: - `RobotState(...)` und `Sensors(...)` werden hier von Hand konstruiert, ohne eine laufende GameEngine — das funktioniert, weil beide reine Datenklassen sind (`data class`, siehe `framework/arena/Models.kt`), also ganz normale Konstruktor-Aufrufe ohne Nebenwirkungen. Das ist der didaktische Kern der Story: `decide()` ist eine reine Funktion (Input `Sensors` -> Output `Action`), daher lässt sie sich isoliert testen, ohne dass ein ganzes Match simuliert werden muss. - `RobotState(id = "bot-0", ...)`: die `id` ist frei gewählt und hat für den Test keine funktionale Bedeutung (der Bot kennt nur `sensors.self/sensors.others`, nicht seine eigene ID direkt) — wichtig ist nur, dass Positionsdaten und Health realistisch sind. - Testfall wählt bewusst gleiches `y` (5 und 5) bei unterschiedlichem `x` (2 und 7) -> Gegner steht östlich in derselben Reihe -> erwartete Aktion ist `Action.Shoot(Direction.EAST)`. Das prüft direkt die `when`-Logik aus 2.2/2.3. - `assertEquals(Action.Shoot(Direction.EAST), action)` funktioniert nur, weil `Action` (bzw. dessen Subtyp `Shoot`) ebenfalls eine `data class/sealed class` mit datenklassen-basierter Gleichheit ist — zwei `Action.Shoot(EAST)`-Instanzen sind laut `equals()` gleich, obwohl es zwei unterschiedliche Objekte im Speicher sind. Guter Punkt, um Schülern `data class`-Equality zu erklären, falls sie fragen, warum `==/assertEquals` hier “einfach funktioniert”. - Package-Deklaration (`package bots.teama`) muss zum tatsächlichen Ordnerpfad passen (Kotlin-Konvention: Package = Verzeichnisstruktur) — sonst Compile-Fehler oder Test wird von Gradle nicht gefunden.

4.2 — Testduell protokollieren

Keine Code-Lösung nötig — organisatorische Story. Hinweis für den Dozenten: einfach in der App zwei Bots auswählen (eigener Bot + ein Bot aus `bots/examples`), Start drücken, Ergebnis im Scoreboard/Log ablesen lassen.

Epic 5

Beide Storys sind rein organisatorisch, keine Musterlösung nötig. Bei 5.1 sicherstellen, dass Teams nicht aus Versehen mehrere widersprüchliche `MeinBot`-Varianten gleichzeitig in `teamXBots` eingetragen lassen, falls sie mehrere Ausbaustufen parallel behalten haben.

Hinweis für den Dozenten (5.1): `teamXBots` ist eine simple `List<RobotBrain>` (siehe `BotRegistry.kt`) — jede zusätzlich eingetragene Instanz taucht separat in der Bot-Auswahl der UI auf. Prüfen, dass die Liste nur die final gewollte(n) Version(en) enthält, nicht z.B. `MeinBot()` und `MeinBotV2()` gleichzeitig, falls das Team iterativ mehrere Klassen angelegt hat statt eine zu überschreiben. `./gradlew build` als Abnahme-Check reicht, um Kompilierfehler auszuschließen — sagt aber nichts über die Spielstärke aus.